

# Software Requirements Specification

**MEA-V2G / FreeV2G**

Reference Implementation for ISO 15118 EV/EVSE

Ruslee Sutthaweeikul

December 22, 2025

# Contents

|          |                                         |           |
|----------|-----------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                     | <b>3</b>  |
| 1.1      | Purpose . . . . .                       | 3         |
| 1.2      | Scope . . . . .                         | 3         |
| 1.3      | Definitions and Acronyms . . . . .      | 3         |
| <b>2</b> | <b>Overall Description</b>              | <b>4</b>  |
| 2.1      | Product Perspective . . . . .           | 4         |
| 2.2      | System Functions . . . . .              | 4         |
| 2.3      | Use Case Diagram . . . . .              | 4         |
| 2.4      | System Architecture . . . . .           | 4         |
| 2.5      | EV State Machine . . . . .              | 4         |
| 2.6      | EVSE State Machine . . . . .            | 5         |
| 2.7      | State Synchronization . . . . .         | 8         |
| 2.8      | Core Components . . . . .               | 8         |
| 2.8.1    | Whitebeet Component . . . . .           | 8         |
| 2.8.2    | EV Component . . . . .                  | 9         |
| 2.8.3    | EVSE Component . . . . .                | 9         |
| 2.8.4    | OCPP Component . . . . .                | 10        |
| 2.9      | Charger Component . . . . .             | 10        |
| 2.9.1    | Implementing a Custom Charger . . . . . | 10        |
| 2.9.2    | TRIO-HP Charger Protocol . . . . .      | 12        |
| 2.10     | Battery Component . . . . .             | 13        |
| 2.11     | Default Charging Routine . . . . .      | 14        |
| 2.11.1   | Default Schedule . . . . .              | 14        |
| 2.11.2   | Physics Simulation Parameters . . . . . | 14        |
| 2.12     | Project File Structure . . . . .        | 14        |
| <b>3</b> | <b>Specific Requirements</b>            | <b>16</b> |
| 3.1      | External Interfaces . . . . .           | 16        |
| 3.1.1    | Communication Interfaces . . . . .      | 16        |
| 3.1.2    | User Interfaces . . . . .               | 16        |
| 3.2      | Functional Requirements . . . . .       | 16        |
| 3.2.1    | FR-01: Protocol Negotiation . . . . .   | 16        |
| 3.2.2    | FR-02: Authorization . . . . .          | 16        |
| 3.2.3    | FR-03: Charging Control . . . . .       | 17        |
| 3.2.4    | FR-04: OCPP Integration . . . . .       | 17        |
| 3.3      | Communication Flow . . . . .            | 17        |
| 3.3.1    | OCPP Sequence Flow . . . . .            | 17        |
| 3.3.2    | CSMS-EVSE-EV Interaction . . . . .      | 18        |
| 3.4      | OCPP Connector State Machine . . . . .  | 18        |

|          |                                          |           |
|----------|------------------------------------------|-----------|
| 3.4.1    | State Transition Logic . . . . .         | 20        |
| 3.4.2    | EVSE to OCPP State Mapping . . . . .     | 21        |
| <b>4</b> | <b>Configuration and Deployment</b>      | <b>23</b> |
| 4.1      | Command Line Interface (CLI) . . . . .   | 23        |
| 4.2      | JSON Configuration . . . . .             | 23        |
| 4.2.1    | EV Configuration (ev.json) . . . . .     | 23        |
| 4.2.2    | EVSE Configuration (evse.json) . . . . . | 23        |
| 4.3      | Software Dependencies . . . . .          | 24        |
| <b>5</b> | <b>Test Plan</b>                         | <b>25</b> |
| 5.1      | Testing Scope . . . . .                  | 25        |
| 5.2      | Test Environment . . . . .               | 25        |
| 5.3      | Automated Test Strategy . . . . .        | 25        |
| 5.3.1    | Unit Testing . . . . .                   | 25        |
| 5.3.2    | Integration Testing . . . . .            | 26        |
| 5.3.3    | System Testing . . . . .                 | 26        |
| 5.3.4    | Test Execution . . . . .                 | 26        |
| 5.3.5    | Initial Test Results . . . . .           | 27        |
| 5.4      | Test Procedures . . . . .                | 27        |
| 5.4.1    | TC-01: System Initialization . . . . .   | 27        |
| 5.4.2    | TC-02: EV Connection . . . . .           | 27        |
| 5.4.3    | TC-03: Authorization . . . . .           | 27        |
| 5.4.4    | TC-04: Charging Loop . . . . .           | 28        |
| 5.4.5    | TC-05: Session Stop . . . . .            | 28        |
| 5.4.6    | TC-06: Safety/Emergency Stop . . . . .   | 28        |
| <b>6</b> | <b>Reference Documents</b>               | <b>29</b> |
| <b>A</b> | <b>OCPP 2.0.1 Protocol Details</b>       | <b>30</b> |
| A.1      | Overview . . . . .                       | 30        |
| A.2      | Core Message Reference . . . . .         | 30        |
| <b>B</b> | <b>Whitebeet Interface Reference</b>     | <b>32</b> |
| B.1      | System & Configuration . . . . .         | 32        |
| B.2      | Control Pilot (CP) . . . . .             | 32        |
| B.3      | SLAC (GreenPHY) . . . . .                | 32        |
| B.4      | V2G Session Management (EVSE) . . . . .  | 33        |
| B.4.1    | Configuration . . . . .                  | 33        |
| B.4.2    | Session Control . . . . .                | 33        |
| B.4.3    | Notifications & Parsing . . . . .        | 33        |

# Chapter 1

## Introduction

### 1.1 Purpose

The purpose of this document is to define the software requirements for the MEA-V2G (FreeV2G) project. This system serves as a reference implementation for controlling the 8 devices WHITE-beet-EI (EVSE) and WHITE-beet-PI (EV) ISO 15118 modules using an Ethernet host control interface (HCI).

### 1.2 Scope

The MEA-V2G software encompasses:

- Parsing of the protocol used to communicate with the WHITE-beet modules.
- Simulation of EVSE and EV behaviors.
- Handling of charging parameters (Voltage, Current, Power).
- Implementation of basic Control Pilot (CP) and SLAC sequences.
- High-level V2G communication management.
- Integration with OCPP 2.0.1 for backend communication.

### 1.3 Definitions and Acronyms

**EV** Electric Vehicle

**EVSE** Electric Vehicle Supply Equipment (Charging Station)

**HCI** Host Control Interface

**SLAC** Signal Level Attenuation Characterization

**V2G** Vehicle-to-Grid

**PNC** Plug & Charge

**EIM** External Identification Means

**OCPP** Open Charge Point Protocol

**CSMS** Charging Station Management System

## Chapter 2

# Overall Description

### 2.1 Product Perspective

The software interacts directly with the WHITE-beet hardware modules via Ethernet or SPI. It acts as the "Host" controller, managing the logic while the WHITE-beet module handles the low-level PLC (Power Line Communication) and ISO 15118 framing.

### 2.2 System Functions

The system enables the following core functions:

1. **Control Pilot (CP) implementation:** Detects EV plugin and manages duty cycles.
2. **SLAC:** Automates the matching process between EV and EVSE at the physical layer.
3. **V2G Session Management:** Handles Service Discovery, Parameter Discovery, and Authorization.
4. **Charging Loop:** Monitoring and control of the active energy transfer.
5. **OCPP Communication:** Interface with a CSMS via OCPP 2.0.1 for remote control and status monitoring.

### 2.3 Use Case Diagram

The following diagram illustrates the primary actors and use cases for the MEA-V2G system.

### 2.4 System Architecture

The system obeys a modular object-oriented architecture. The core logic handles the state machines for EV and EVSE, while the 'Whitebeet' class abstracts the hardware communication.

### 2.5 EV State Machine

The EV internal logic follows a distinct state machine to manage the charging session lifecycle, as observed in the implementation.

**Init** Initial state where the EV controller is started and hardware interfaces (SPI/Ethernet) are initialized.

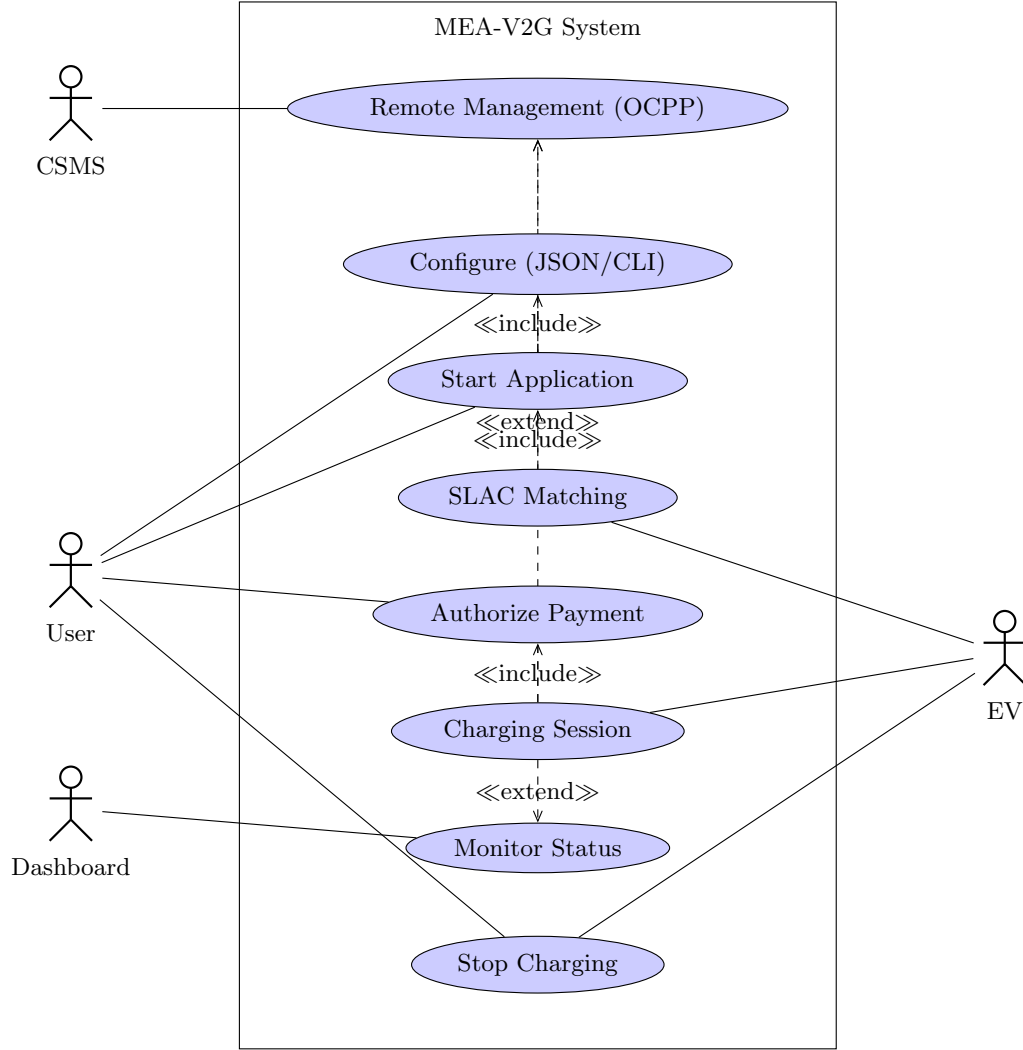


Figure 2.1: MEA-V2G High-Level Use Case Diagram

**SessionStarted** The V2G session has been established. The EV has successfully matched with the EVSE via SLAC.

**CableCheck** The EV waits for the EVSE to perform the cable check (isolation test) to ensure safety.

**PreCharging** Pre-charge phase where the EV matches its battery voltage to the DC bus voltage.

**Charging** The main state where energy transfer occurs. The EV sends *CurrentDemand* requests in a loop.

**ChargingStopped** The session is terminated, either normally (target SOC reached) or due to an error.

## 2.6 EVSE State Machine

The EVSE logic complements the EV, acting as the server component that handles session setup, authorization, and power delivery control.

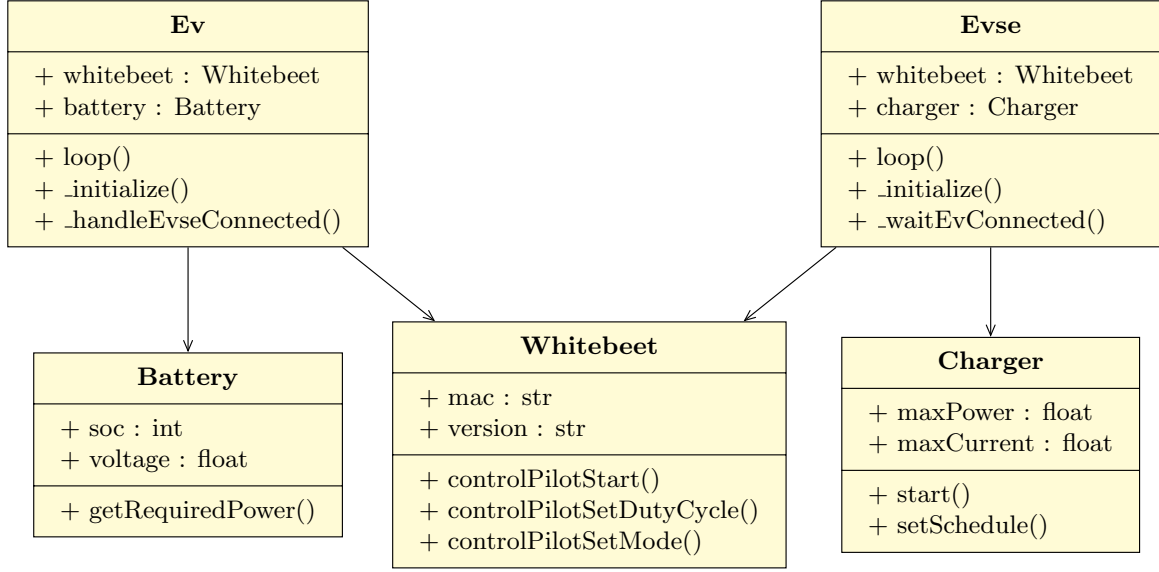


Figure 2.2: MEA-V2G Simplified Class Diagram

**Init** Initialization of the Whitebeet module services (CP, SLAC, V2G).

**Wait** The EVSE waits for an EV to physically plug in (Control Pilot State B).

**SLAC** Signal Level Attenuation Characterization process to pair the EV and EVSE.

**Session** High-level communication session established; protocol negotiation occurs.

**Authorization** Verifying if the EV is authorized to charge (via EIM or Plug & Charge).

**Params** Exchanging charge parameter limits (Maximum Voltage, Current, Power limits).

**Cable** Performing cable isolation check to ensure no faults exist.

**PreCharge** Adjusting DC output voltage to match the EV's reported battery voltage.

**Charge** Active energy transfer loop responding to the EV's current demand.

**Stop** Session termination and welding detection sequence.

**Faulted** Critical error state (e.g., GFDI trip, Overcurrent). The EVSE opens contactors and sends a `StatusNotification(Faulted)` to the CSMS via OCPP. It awaits a manual or remote reset.

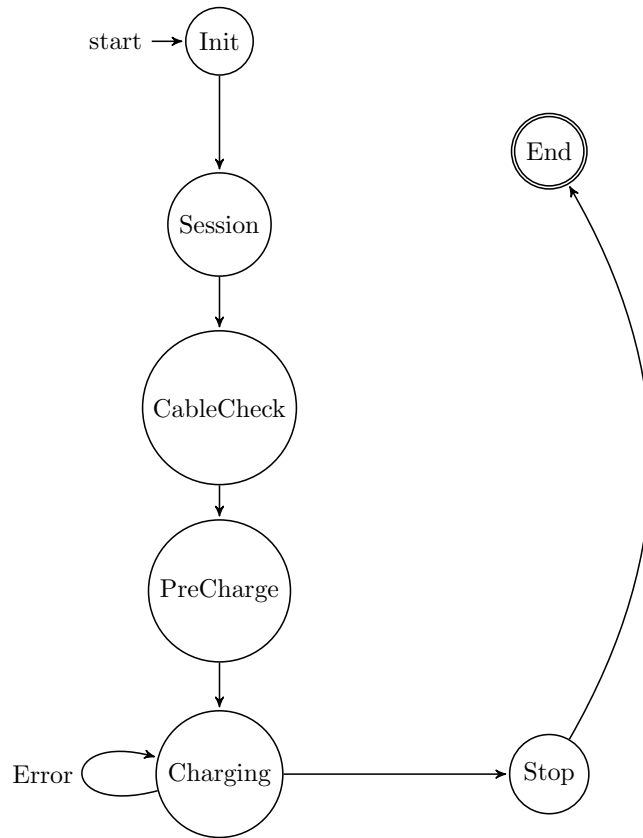


Figure 2.3: EV State Machine

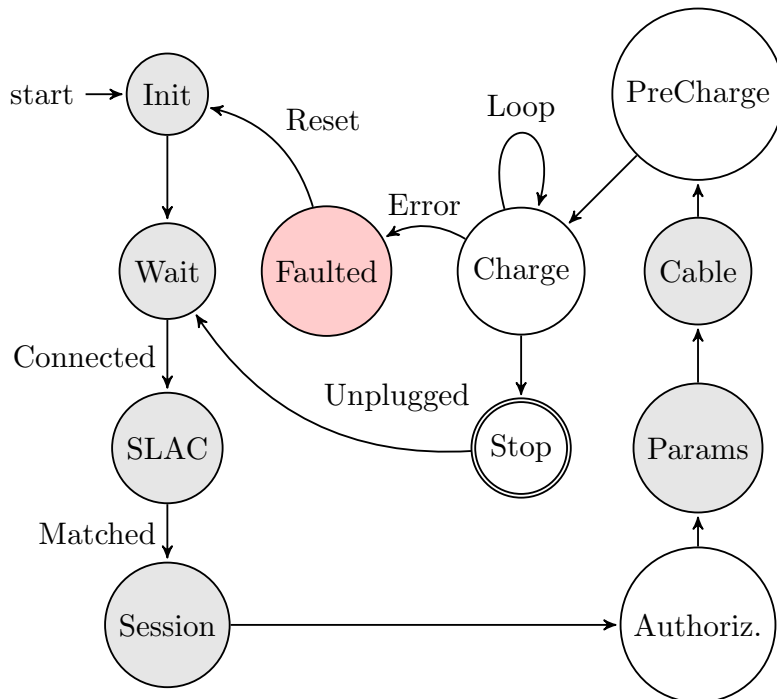


Figure 2.4: EVSE State Machine



## 2.7 State Synchronization

The charging process relies on the tight synchronization between the EV and EVSE states. Figure 2.5 illustrates the parallel state progression and key synchronization points.

**Discovery** Control Pilot signal detection (State A  $\rightarrow$  B).

**Setup** SLAC matching and V2G Session Setup.

**Safety** Both sides must confirm safety parameters during Cable Check.

**Sync** Voltage levels must be synchronized during Pre-Charge before the contactors close.

**Power** The Charging Loop is a lock-step process where the EV requests and the EVSE delivers.

**Term** Coordinated shutdown to ensure contactors open safely (Welding Detection).

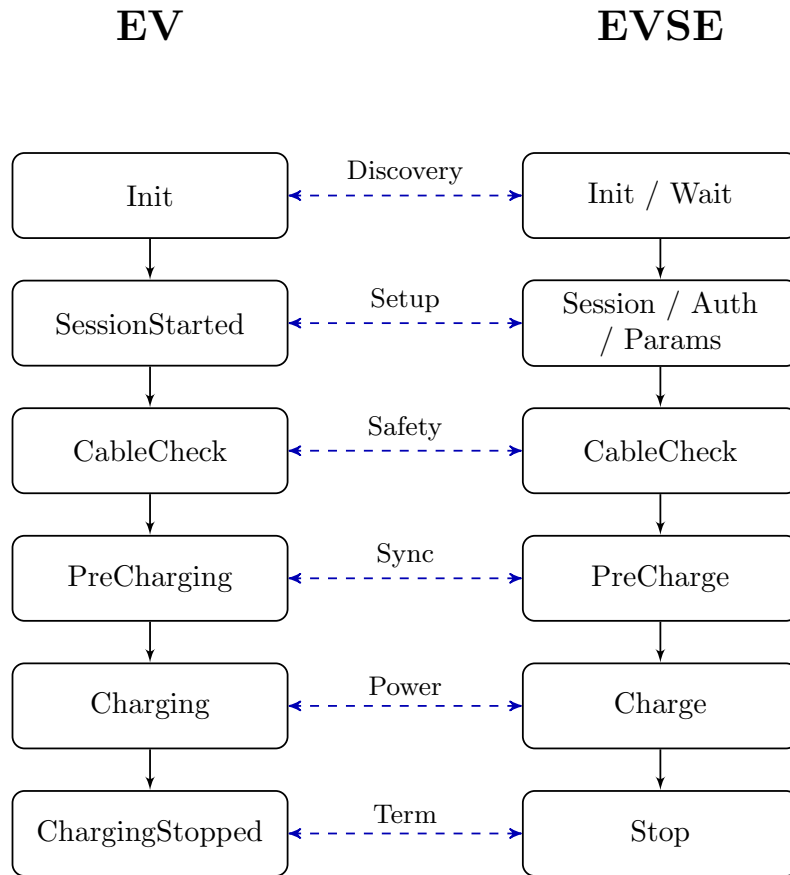


Figure 2.5: EV-EVSE State Synchronization

## 2.8 Core Components

The system comprises the following key software components:

### 2.8.1 Whitebeet Component

The **Whitebeet** class serves as the Hardware Abstraction Layer (HAL). It manages the low-level communication with the physical ISO 15118 module via Ethernet or SPI.

- **Packet Framing:** Handles the encapsulation of Host Control Interface (HCI) commands.
- **Event Dispatching:** Receives raw data from the hardware and dispatches events (e.g., EV\_CONNECTED, V2G\_MESSAGE) to the higher-level logic.
- **Service Management:** Exposes methods to control Control Pilot (CP) state, SLAC, and V2G messaging.

## Protocol Structure

The Whitebeet module communicates via a serial interface using a framed protocol. Each frame is structured as follows:

| Start | Mod ID | Sub ID | Req ID | Len (2)    | Payload        | CRC    | End  |
|-------|--------|--------|--------|------------|----------------|--------|------|
| 0xC0  | 1 Byte | 1 Byte | 1 Byte | Big Endian | <i>N</i> Bytes | 1 Byte | 0xC1 |

## HCI State Transition Mapping

The following table maps the critical EVSE states to the corresponding ISO 15118 V2G messages and the Whitebeet HCI notifications that trigger the transitions.

| EVSE State        | ISO 15118 Message           | HCI Notification      | HCI ID |
|-------------------|-----------------------------|-----------------------|--------|
| <b>CableCheck</b> | ChargeParameterDiscoveryReq | SessionStarted        | 0x80   |
| Authorization     | AuthorizationReq            | RequestAuthorization  | 0x82   |
| <b>PreCharge</b>  | PowerDeliveryReq(Start)     | PowerDeliveryReq      | 0x8E   |
| <b>PreCharge</b>  | PreChargeReq                | PreChargeStarted      | 0x88   |
| <b>Charge</b>     | CurrentDemandReq            | DCChargeParamsChanged | 0x85   |
| Stop              | PowerDeliveryReq(Stop)      | RequestStopCharging   | 0x8A   |
| <b>Stop</b>       | SessionStopReq              | SessionStopped        | 0x8C   |
| Faulted           | (Any Error)                 | SessionError          | 0x8E   |

Table 2.1: Whitebeet HCI State Mapping

### 2.8.2 EV Component

The EV class is the central controller for the client side. It orchestrates the charging session from the vehicle's perspective.

- **State Machine:** Implements the ISO 15118 client flow (Session Setup → Charge Parameter Discovery → Power Delivery).
- **Hardware Aggregation:** Owning instance of **Whitebeet** (for comms) and **Battery** (for energy simulation).

### 2.8.3 EVSE Component

The EVSE class is the central controller for the station side. It manages the server-side logic and power delivery authorization.

- **State Machine:** Implements the ISO 15118 server flow, including Service Discovery and Plug & Charge authorization.
- **Hardware Aggregation:** Owning instance of **Whitebeet** (for comms) and **Charger** (for power delivery simulation).
- **Scheduling:** Manages SASchedules based on configuration.

### 2.8.4 OCPP Component

The system integrates an OCPP 2.0.1 client to enable remote management.

- **OcppInterface:** Adapts the `ocpp` library's `ChargePoint` class to the MEA-V2G architecture. It maps OCPP commands (e.g., `RequestStartTransaction`) to local `Charger` actions.
- **OcppWorker:** A dedicated thread that manages the `asyncio` event loop and WebSocket connection to the CSMS, ensuring non-blocking operation alongside the main application loop.

## 2.9 Charger Component

The `Charger` class performs a physical simulation of the power electronics, ensuring realistic behavior during the charging session. Its key functions include:

- **Output Ramping:** Instead of instantaneous jumps, the charger simulates the physical "slewing" of voltage and current over time. It uses defined  $\Delta U$  and  $\Delta I$  rates to ramp the output toward the target setpoint.
- **Limit Enforcement:** It strictly enforces hard safety limits for the system (e.g., `evse_max_current`, `evse_max_power`), protecting the modeled hardware.
- **Setpoint Control:** The EV provides dynamic target values (*Target Voltage*, *Target Current*), which the charger attempts to follow within its safety constraints.
- **Fault Detection:** The class includes methods like `isVoltageLimitExceeded()` and `isPowerLimitExceeded()` to trigger immediate session termination if operating conditions are violated.

The `Charger` class implements the `ChargerInterface` abstract base class. This design pattern allows the simulation module to be easily swapped with a real hardware driver, such as the `CanCharger` class, which communicates with physical power electronics via CAN bus using the `python-can` library, while maintaining a consistent API for the EVSE logic.

### 2.9.1 Implementing a Custom Charger

To integrate specific power electronics hardware, developers must create a subclass of `ChargerInterface`. This ensures the EVSE logic remains decoupled from the physical layer.

A custom implementation must override the abstract methods to map high-level commands to hardware-specific protocols. The following sections detail the requirements for each group of methods.

#### Lifecycle Management

These methods control the high-level operational state of the power electronics.

**start()** Called when the EVSE initializes or prepares for a session. It should initialize hardware communication (e.g., open CAN sockets). **Note:** This method should **not** automatically close the output contactors; that is handled by the state machine after safe voltage matching.

**stop() Critical Safety Method.** Called to terminate a session or during error conditions. It must immediately disable power output, open contactors, and return the hardware to a safe idle state (0V/0A).

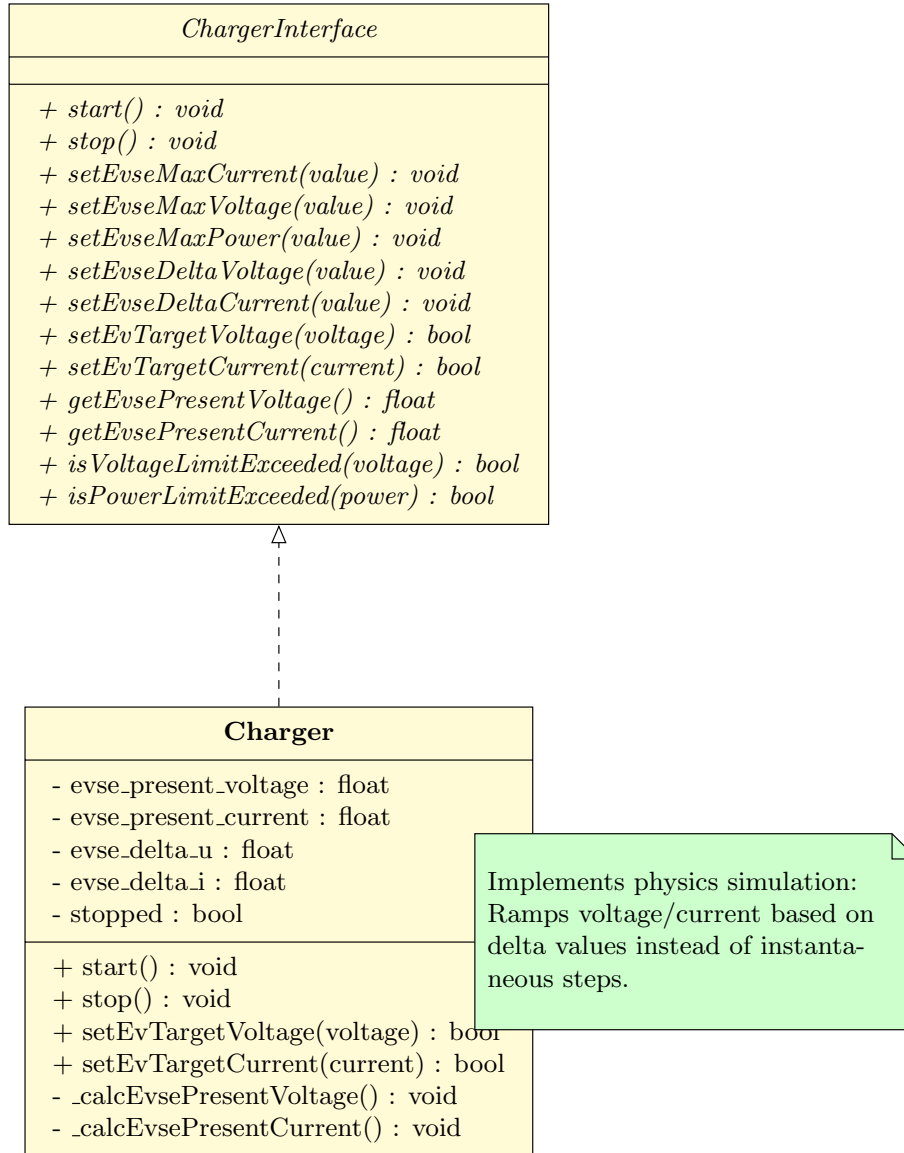


Figure 2.6: Charger Interface and Implementation

### Control Loop Methods

These methods are called cyclically (typically every 10-100ms) during the active charging phase.

**setEvTargetVoltage(val)** Receives the EV's requested voltage setpoint (in Volts). The implementation must transmit this request to the power supply unit (PSU).

**setEvTargetCurrent(val)** Receives the EV's requested current setpoint (in Amps). The implementation must ensure the PSU imposes this current limit.

*Return Value:* These methods must return **True** if the command was successfully queued or sent to the hardware, and **False** if the request triggers a local hardware fault or limit violation.

### Telemetry Methods

The EVSE state machine relies on accurate feedback to perform safety checks (e.g., Welding Detection) and process steps (e.g., Pre-Charge).

`getEvsePresentVoltage()` Must return the *measured* voltage at the DC output terminals. Do not return the internal setpoint; the actual value is required to verify contactor states.

`getEvsePresentCurrent()` Must return the *measured* current flowing to the EV.

### Configuration Accessors

The interface includes getters and setters for electrical limits (e.g., `setEvseMaxCurrent`, `getEvseMaxPower`).

- For a **Simulation** (like `Charger.py`), these methods simply store and retrieve variables to emulate different hardware capabilities.
- For **Physical Hardware**, the setters (e.g., `setEvseMaxCurrent`) should theoretically configure the hardware's internal safety latches if supported. The getters must return the valid limits to be advertised to the EV during the *Charge Parameter Discovery* phase.

## 2.9.2 TRIO-HP Charger Protocol

This section details the specific hardware and communication protocol for the TRIO-HP/3AC/1KDC/20KW/BI power module, which is supported by the system via a CAN bus interface.

### Hardware Description

- **Brand:** PHOENIX CONTACT
- **Model:** TRIO-HP/3AC/1KDC/20KW/BI
- **Function:** Bidirectional DC/AC and AC/DC converter.
- **Communication:** CAN 2.0B Extended Frame (29-bit Identifier).
- **Baud Rate:** 125 kbps.
- **Termination:** 120  $\Omega$  resistors required.

### Protocol Structure

The protocol uses a 29-bit extended identifier structured as follows:

| Error (3 bits) | Device No. (4 bits) | Command (6 bits) | Target (8 bits) | Source (8 bits) |
|----------------|---------------------|------------------|-----------------|-----------------|
| 28-26          | 25-22               | 21-16            | 15-8            | 7-0             |

- **Error Code:** 0x00 = Normal.
- **Device No:** 0x0A for individual modules, 0x0B for groups.
- **Target/Source Address:** Unique module ID; Controller default is 0xF0.

### Key Commands

The system utilizes the following CAN commands to control the charger:

| Description      | Cmd ID | Data Bytes (Hex) | Notes                            |
|------------------|--------|------------------|----------------------------------|
| Read DC Voltage  | 0x23   | 10 01 00 00 ...  | Response contains voltage in mV. |
| Read DC Current  | 0x23   | 10 02 00 00 ...  | Response contains current in mA. |
| Set DC Voltage   | 0x24   | 10 01 00 00 ...  | Sets output voltage target (mV). |
| Set DC Current   | 0x24   | 10 02 00 00 ...  | Sets output current limit (mA).  |
| Enable Operation | 0x24   | 11 10 00 00 ...  | Byte 7: 0xA0 (On), 0xA1 (Off).   |

## Protocol Integration

The following table maps the abstract EVSE states and OCPP events to specific CAN protocol actions, ensuring the hardware behaves correctly during the session lifecycle.

| EVSE State       | Event / Trigger           | System Action                                       | CAN Command Sequence                                                            |
|------------------|---------------------------|-----------------------------------------------------|---------------------------------------------------------------------------------|
| <b>PreCharge</b> | Session Setup             | Match EV battery voltage to prevent inrush current. | Set DC Voltage (Target)<br>Read DC Voltage (Verify)<br>Enable Operation (0xA0)  |
| <b>Charge</b>    | SetChargingProfile (OCPP) | Apply new maximum current limit from CSMS.          | Set DC Current (Profile Limit)                                                  |
| <b>Charge</b>    | CurrentDemand (EV)        | Follow EV's dynamic current request.                | Set DC Current (EV Target)                                                      |
| <b>Stop</b>      | Session Check / User Stop | Safe shutdown sequence.                             | Set DC Current (0 A)<br>Read DC Current (Verify 0A)<br>Disable Operation (0xA1) |

## CanCharger Implementation

The **CanCharger** class provides the concrete implementation for the TRIO-HP protocol. It utilizes the **python-can** library to handle low-level bus arbitration and message framing.

- **Bus Abstraction:** Supports multiple interfaces (SocketCAN, Kvaser, Virtual) via the library's abstraction layer.
- **Threaded Listening:** A continuous background thread monitors the bus for broadcast updates (e.g., current system voltage) to keep the internal state synchronized with the hardware.
- **Error Handling:** Automatically detects connection failures or missing libraries, falling back to safe states or raising initialization errors.

## 2.10 Battery Component

The **Battery** class provides a logical model of the EV's energy storage, managing State of Charge (SOC) and physical limits.

- **Energy Tracking:** Maintains the battery capacity (Wh) and current energy level. SOC is dynamically calculated as  $\frac{\text{Current Level}}{\text{Capacity}} \times 100$ .
- **Charging Process:** The `tickSimulation()` method accumulates energy over time intervals based on the instantaneous input voltage and current ( $E = V \times I \times \Delta t$ ).
- **Configurable Limits:** Supports defining maximum Voltage, Current, and Power constraints specific to the chosen Energy Transfer Mode (AC vs. DC).
- **Status Reporting:** Continuously updates the SOC and charging status ('is\_full', 'is\_charging'), which are reported back to the EVSE via ISO 15118 messages.

## 2.11 Default Charging Routine

In the absence of an external configuration file, the system defaults to a hardcoded "Simple Charging Routine" defined in `Application.py`. This routine ensures the EVSE behaves predictably during initial testing or standalone operation.

### 2.11.1 Default Schedule

The default charging profile is a time-dependent power curve that simulates a decreasing power availability (e.g., grid balancing or solar curve):

- **0 to 30 minutes:** Maximum Power 25,000 W.
- **30 to 60 minutes:** Maximum Power reduced to 18,750 W.
- **Over 60 minutes:** Maximum Power further reduced to 12,500 W.

### 2.11.2 Physics Simulation Parameters

To simulate realistic hardware behavior, the `Charger` class uses specific delta parameters to ramp voltage and current rather than applying instantaneous steps:

- **Delta Voltage:** 0.5 V per tick.
- **Delta Current:** 0.05 A per tick.
- **Max Limits:** 400 V / 100 A / 25 kW.

## 2.12 Project File Structure

The following table summarizes the key files in the repository and their specific purposes within the architecture:

| File Name           | Description                                                                                                                           |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Application.py      | The main entry point. Parses command-line arguments and initializes the <b>EV</b> or <b>EVSE</b> instance based on the selected role. |
| Ev.py               | Implements the Electric Vehicle (client) logic, including the ISO 15118 state machine and session management.                         |
| Evse.py             | Implements the Charging Station (server) logic, managing service discovery, authorization, and power delivery.                        |
| Whitebeet.py        | The Hardware Abstraction Layer (HAL). Handles low-level communication with the Codico Whitebeet module.                               |
| Charger.py          | A physics simulation class for the EVSE's power electronics. Simulates voltage/current ramping and enforces safety limits.            |
| ChargerInterface.py | An abstract base class defining the standard API for charger implementations, enabling pluggable drivers (Simulation vs. CAN).        |
| Battery.py          | A logical model of the EV battery, tracking State of Charge (SOC) and energy capacity.                                                |
| EthernetAdapter.py  | Driver for communicating with the Whitebeet module via a network socket.                                                              |
| SpiAdapter.py       | Driver for communicating with the Whitebeet module via the SPI bus (e.g., on Raspberry Pi).                                           |
| FramingInterface.py | Defines the packet structure and framing rules for the Host Control Interface (HCI) protocol.                                         |
| api_server.py       | A lightweight Flask server that exposes internal state (like SOC, current power) via a REST API for monitoring.                       |
| gui_launcher.py     | A graphical utility to easily launch <b>Application.py</b> without using the command line.                                            |
| ev.json / evse.json | Configuration files defining capabilities, limits, and schedules for the respective roles.                                            |



## Chapter 3

# Specific Requirements

### 3.1 External Interfaces

#### 3.1.1 Communication Interfaces

- **Ethernet:** Main control interface for sending HCI commands and receiving notifications.
- **SPI:** Alternative control interface (e.g., for Raspberry Pi integration).
- **Ocpp Interface:** WebSocket connection (JSON over WebSockets) to a central management system (CSMS).

#### 3.1.2 User Interfaces

- **CLI (Command Line Interface):** Primary mode of operation. Outputs session logs and accepts interactive input (e.g., authorization "yes/no").
- **GUI:** A graphical launcher ('gui\_launcher.py') provided for configuration and startup.
- **API:** A REST API (default port 5000) for real-time status monitoring (e.g., for Grafana dashboards).

### 3.2 Functional Requirements

#### 3.2.1 FR-01: Protocol Negotiation

The system shall support negotiation of ISO 15118 protocol versions.

- The EVSE shall advertise supported protocols.
- The EV shall select a mutually supported protocol (e.g., DIN SPEC 70121 or ISO 15118-2).

#### 3.2.2 FR-02: Authorization

The system shall support multiple authorization modes:

- **EIM (External Identification Means):** Manual authorization via console input.
- **PnC (Plug & Charge):** Automated certificate-based authorization.

### 3.2.3 FR-03: Charging Control

During the charging loop, the system shall:

- Exchange charging parameters (SOC, Target Voltage, Target Current).
- Terminate the session upon "Stop Charging" request from either side.

### 3.2.4 FR-04: OCPP Integration

The system shall implement OCPP 2.0.1 features:

- Connect to a CSMS via WebSocket.
- Send `BootNotification` and `Heartbeat` messages.
- Accept `RequestStartTransaction` and `RequestStopTransaction` commands from the CSMS.
- Support `SetVariables` to configure EVSE limits remotely.

## 3.3 Communication Flow

The following sequence diagram depicts the simplified ISO 15118 charging loop interaction between the EV and EVSE, which is the core logical flow of the application.

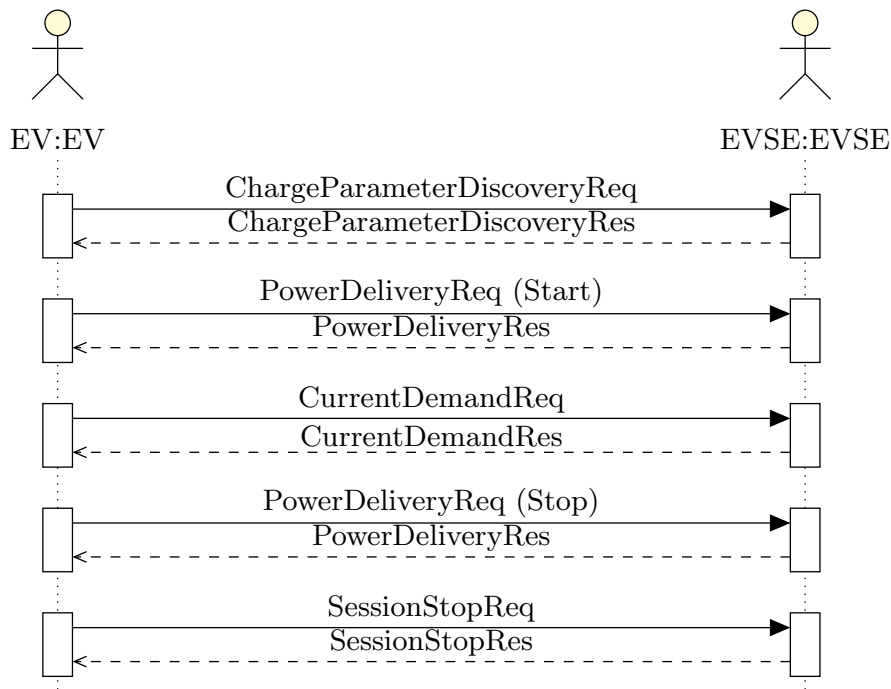


Figure 3.1: ISO 15118 Charging Loop Sequence

### 3.3.1 OCPP Sequence Flow

The following diagram illustrates the interaction between the EVSE (Charge Point) and the CSMS (Charging Station Management System) via OCPP 2.0.1.

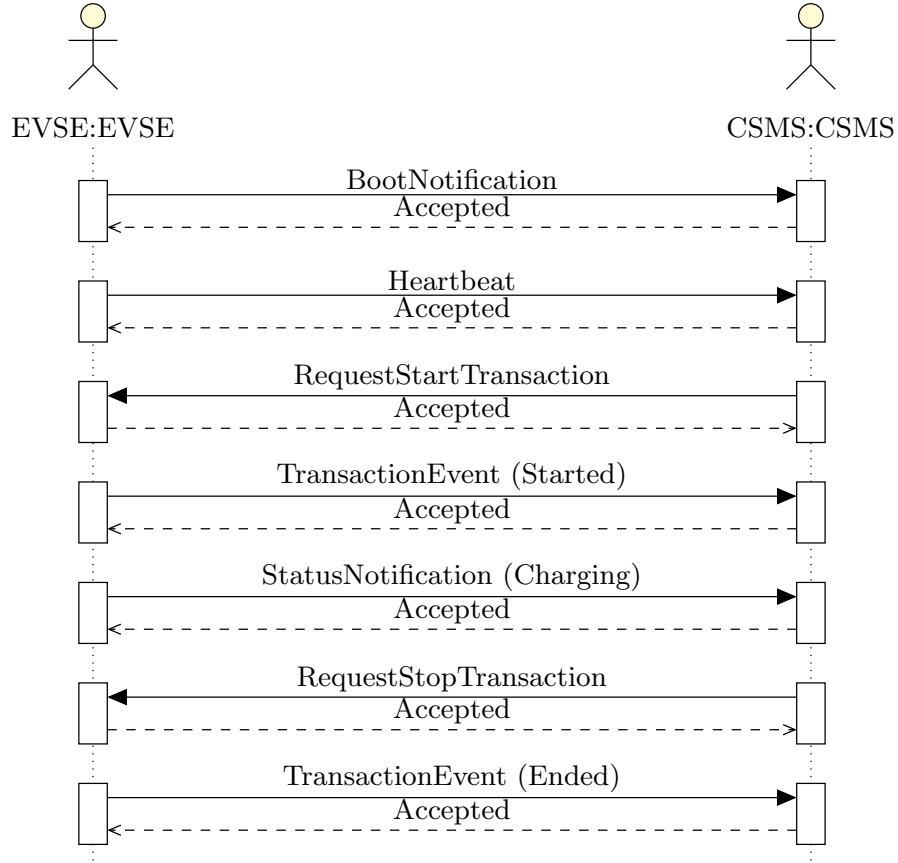


Figure 3.2: OCPP 2.0.1 Interaction Sequence

### 3.3.2 CSMS-EVSE-EV Interaction

This diagram focuses on the 3-tier communication flow involving the management system, the charging station, and the vehicle. It illustrates a scenario where a remote command initiates a session with smart charging limits.

## 3.4 OCPP Connector State Machine

The EVSE maintains a logical state for each connector, which is reported to the CSMS via Status Notification messages. This state machine abstracts the underlying ISO 15118 physical processes.

**Available** The connector is free and ready for a new session.

**Preparing** An EV is plugged in or an authorization process is pending.

**Charging** Active energy transfer is in progress.

**SuspendedEV** Charging is paused by the EV (e.g., battery full).

**SuspendedEVSE** Charging is paused by the EVSE (e.g., smart charging limit).

**Finishing** The session has stopped, waiting for the plug to be removed.

**Reserved** The connector is reserved for a specific user.

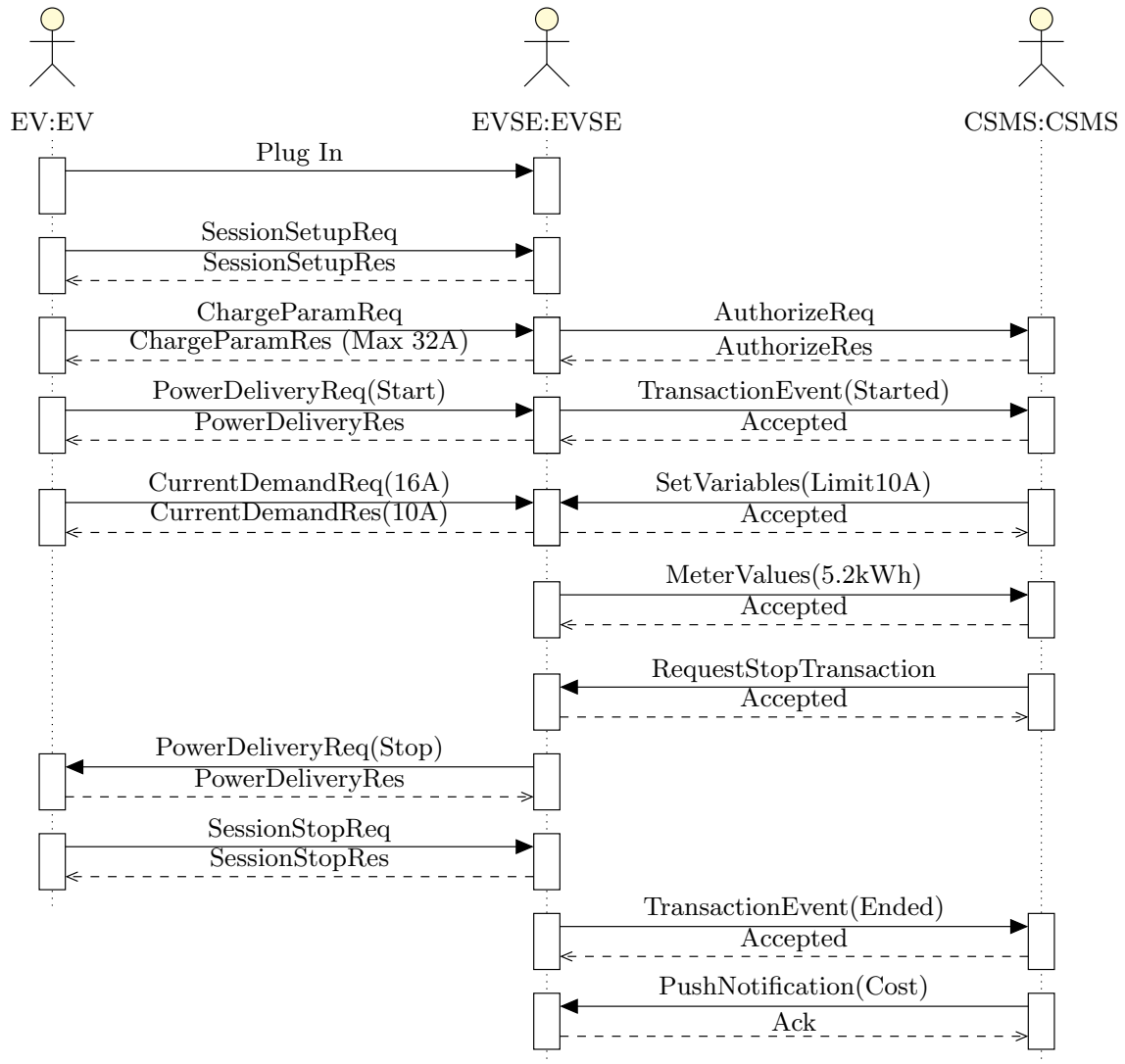


Figure 3.3: 3-Tier (CSMS-EVSE-EV) Sequence Flow

**Unavailable** The connector is operative but not available for use (e.g., maintenance).

**Faulted** A critical error has occurred.

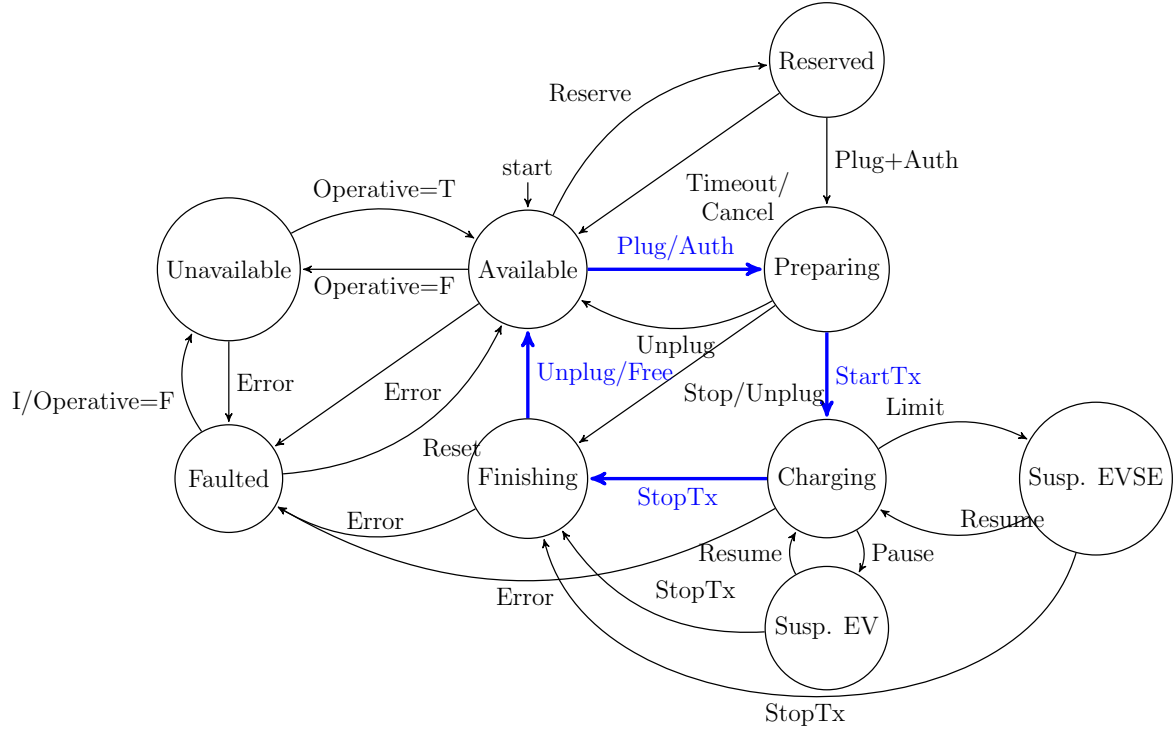


Figure 3.4: OCPP Connector Status State Machine

### 3.4.1 State Transition Logic

The following table details the communication packets exchanged between the EV, EVSE, and CSMS that trigger the transitions in the OCPP Connector State Machine.

| Transition                            | Physical Trigger                             | ISO 15118 (EV-EVSE)                       | OCPP (EVSE-CSMS)                                        |
|---------------------------------------|----------------------------------------------|-------------------------------------------|---------------------------------------------------------|
| <i>Normal Operation</i>               |                                              |                                           |                                                         |
| Available → Preparing                 | Plug In (CP $A \rightarrow B$ )              | SLAC Match, SessionSetupReq               | StatusNotification(Preparing)                           |
| Preparing → Charging                  | Power Delivery Start (CP $B \rightarrow C$ ) | PowerDeliveryReq(Start), CurrentDemandReq | TransactionEvent(Started), StatusNotification(Charging) |
| Charging → Finishing                  | Session Stop (CP $C \rightarrow B$ )         | PowerDeliveryReq(Stop) or SessionStopReq  | TransactionEvent(Ended), StatusNotification(Finishing)  |
| Finishing → Available                 | Unplug (CP $B \rightarrow A$ )               | N/A                                       | StatusNotification(Available)                           |
| <i>Suspended States</i>               |                                              |                                           |                                                         |
| Charging → SuspendedEV                | EV pauses (e.g. max SOC)                     | CurrentDemandReq (Target=0A)              | StatusNotification(SuspendedEV)                         |
| SuspendedEV → Charging                | EV resumes                                   | CurrentDemandReq (Target>0A)              | StatusNotification(Charging)                            |
| Charging → SuspendedEVSE              | EVSE limits power                            | CurrentDemandRes (MaxCurrent=0A)          | StatusNotification(SuspendedEVSE)                       |
| SuspendedEVSE → Charging              | EVSE lifts limit                             | CurrentDemandRes (MaxCurrent>0A)          | StatusNotification(Charging)                            |
| Suspended → Finishing                 | Stop during pause                            | SessionStopReq                            | TransactionEvent(Ended), StatusNotification(Finishing)  |
| <i>Reservation &amp; Availability</i> |                                              |                                           |                                                         |
| Available → Reserved                  | N/A                                          | N/A                                       | ReserveNowReq, StatusNotification(Reserved)             |
| Reserved → Preparing                  | Plug In + Auth                               | SessionSetupReq                           | StatusNotification(Preparing)                           |
| Reserved → Available                  | Timeout / Cancel                             | N/A                                       | CancelReservationReq, StatusNotification(Available)     |
| Available → Unavailable               | Operator Command                             | N/A                                       | ChangeAvailabilityReq, StatusNotification(Unavailable)  |
| Unavailable → Available               | Operator Command                             | N/A                                       | ChangeAvailabilityReq, StatusNotification(Available)    |
| <i>Exception Layout</i>               |                                              |                                           |                                                         |
| Preparing → Available                 | Unplug (Invalid Session)                     | SessionStopReq                            | StatusNotification(Available)                           |
| Any → Faulted                         | Error / Fault                                | Welding Detection Fail, etc.              | StatusNotification(Faulted)                             |
| Faulted → Available                   | Reset / Recovery                             | N/A                                       | ResetReq, StatusNotification(Available)                 |
| Faulted → Unavailable                 | Permanent Fault                              | N/A                                       | ChangeAvailabilityReq, StatusNotification(Unavailable)  |

### 3.4.2 EVSE to OCPP State Mapping

The following table defines how the internal ISO 15118 EVSE states map to the reported OCPP Connector Status.

| <b>EVSE State (ISO 15118)</b> | <b>OCPP Status</b> | <b>Notes</b>                                        |
|-------------------------------|--------------------|-----------------------------------------------------|
| Init                          | Unavailable        | System initialization or re-boot.                   |
| Wait                          | Available          | Ready for a new session (CP State A).               |
| Wait (Reserved)               | Reserved           | Reserved for specific idTag via <i>ReserveNow</i> . |
| SLAC                          | Preparing          | Plugged in, matching physics.                       |
| Session, Authorization        | Preparing          | Protocol negotiation and Auth.                      |
| Params, Cable                 | Preparing          | Parameter discovery and Cable Check.                |
| PreCharge                     | Charging           | Contactors closing, voltage matching.               |
| Charge                        | Charging           | Active energy transfer.                             |
| Charge (CurrentDemand=0)      | SuspendedEV        | EV is not drawing power.                            |
| Charge (limit=0)              | SuspendedEVSE      | EVSE is preventing power flow.                      |
| Stop                          | Finishing          | Session ended, waiting for unplug.                  |
| (Any Error)                   | Faulted            | Critical hardware or protocol failure.              |

## Chapter 4

# Configuration and Deployment

### 4.1 Command Line Interface (CLI)

The system is launched via `Application.py` with several arguments to define its operating mode:

- `-r, -role`: Specifies the mode of operation (**EV** or **EVSE**).
- `-i, -interface`: Defines the network interface connected to the Whitebeet module (e.g., `eth0`).
- `-c, -config`: Path to the EV configuration file (default: `ev.json`).
- `-ec, -evse-config`: Path to the EVSE configuration file (default: `evse.json`).
- `-auto`: (EVSE only) Enables automatic authorization for Plug & Charge.
- `-api-port`: Sets the port for the REST API status monitor.
- `-ocpp-url`: (EVSE only) WebSocket URL for the OCPP CSMS.
- `-ocpp-id`: (EVSE only) The Charge Point ID to Identify the station to the CSMS.

### 4.2 JSON Configuration

Behavioral parameters are externalized in JSON files to allow run-time customization without code changes.

#### 4.2.1 EV Configuration (`ev.json`)

Defines the characteristics of the Electric Vehicle:

- **ev**: Supported protocols (ISO 15118-2, DIN), payment methods (EIM, PnC), and energy transfer modes.
- **battery**: Physical parameters including capacity, maximum voltage/current, and initial SOC.

#### 4.2.2 EVSE Configuration (`evse.json`)

Defines the characteristics of the Charging Station:

- **charger**: Output limits (Max Voltage/Current/Power) and slew rates ( $\Delta V, \Delta I$ ) for the simulation.



- **schedule**: Defines power delivery schedules (SASchedules) sent to the EV during the Charge Parameter Discovery phase.

### 4.3 Software Dependencies

The MEA-V2G software relies on the following external Python libraries for hardware communication and protocol handling:

- **python-can**: Required for the **CanCharger** class to communicate with CAN-based power modules.
- **spidev**: Required for SPI communication with the Whitebeet module on Raspberry Pi platforms.
- **RPi.GPIO**: Required for General Purpose I/O control on Raspberry Pi.
- **Flask**: Required for the REST API server.

# Chapter 5

## Test Plan

### 5.1 Testing Scope

The validation of the MEA-V2G system ensures compliance with ISO 15118 standards and robust handling of high-voltage safety.

1. **Unit Testing:** Verification of individual classes (e.g., ‘Battery’, ‘Charger’) using the Python ‘unittest’ framework.
2. **Integration Testing:** Verification of the Host Control Interface (HCI) between the application and the Whitebeet module.
3. **System Testing:** End-to-end reliability provided by pairing a simulated EV with the EVSE.

### 5.2 Test Environment

- **DUT (Device Under Test):** The MEA-V2G software running on a Linux host (e.g., Raspberry Pi 4).
- **Hardware Setup:**
  - 1x Codico Whitebeet-EI (EVSE Role)
  - 1x Codico Whitebeet-PI (EV Role) connected via PLC coupling.
  - Physical or Simulated Power Electronics.

### 5.3 Automated Test Strategy

The project employs a tiered automated testing strategy using the Python ‘unittest’ framework. This ensures regressions are caught early and that individual components behave correctly before being integrated.

#### 5.3.1 Unit Testing

Unit tests isolate specific classes and methods, mocking out external dependencies like hardware interfaces and network libraries.

`tests/unit/test_charger.py` Verifies the **Charger** physics simulation.

- Validates voltage and current ramping logic ( $dV/dt$ ,  $dI/dt$ ).

- Ensures safety limits (Max Voltage/Current/Power) are enforced.
- Checks start/stop behavior.

`tests/unit/test_whitebeet.py` Verifies the `Whitebeet` HAL module.

- Mocks the internal `FramingInterface` and `Logger`.
- Validates command construction for SLAC and V2G modes.
- Checks error handling for invalid input parameters.

`tests/unit/test_ocpp.py` Verifies the `OcppInterface`.

- Mocks the `ocpp` library to avoid heavy dependencies in CI environments.
- Tests `BootNotification` handling (Accepted/Rejected).
- Tests `RemoteStartTransaction` and `RemoteStopTransaction` flows.

### 5.3.2 Integration Testing

Integration tests focus on the interaction between the application logic and the hardware drivers (mocked).

`tests/integration/test_evse.py` Verifies the `Evse` class orchestrator.

- Simulates the complete EV connection flow (CP State change → SLAC → V2G Session).
- Validates that V2G messages (e.g., `ChargeParameterDiscovery`) correctly update the `Charger` setpoints.

### 5.3.3 System Testing

System tests verify the full end-to-end stack, including the external management interface.

`tests/system/test_system.py` Verifies the connection between OCPP and the local Charging hardware.

- Validates that an OCPP `RemoteStart` command successfully transitions the local `Charger` to an active state.
- Verifies that hardware faults or stops are correctly reported back to the OCPP layer.

### 5.3.4 Test Execution

The entire test suite can be executed from the project root directory using the standard Python test runner.

```
# Run all tests (Unit, Integration, System)
python3 -m unittest discover tests
```

```
# Run only Unit Tests
python3 -m unittest discover tests/unit
```

```
# Run only Integration Tests
python3 -m unittest discover tests/integration
```

```
# Run in Verbose Mode (shows individual test names)
python3 -m unittest discover tests -v
```

### 5.3.5 Initial Test Results

As of December 2025, the initial automated test suite consisting of 20 tests has been executed successfully.

- **Unit Tests:** 15 tests passed. Covers ‘Charger’ logic, ‘Whitebeet’ protocol framing, and ‘OcppInterface’ messages.
- **Integration Tests:** 3 tests passed. Validates ‘Evse’ orchestrator connection flows and parameter handling.
- **System Tests:** 2 tests passed. Validates end-to-end OCPP remote start/stop commands.

**Result:** 100% Pass Rate (20/20). Execution time < 0.3 seconds.

## 5.4 Test Procedures

### 5.4.1 TC-01: System Initialization

**Objective:** Verify the application correctly detects and initializes the Whitebeet hardware.

**Steps:**

1. Power on the detailed hardware.
2. Launch ‘python3 Application.py –role evse’.
3. Observe console logs for ‘[Whitebeet] System Version: x.y.z’.

**Expected Result:** The system reports a valid firmware version and enters the ‘Wait’ state (CP State A).

### 5.4.2 TC-02: EV Connection

**Objective:** Verify successful plug-in detection and SLAC matching.

**Steps:**

1. Ensure EVSE is in ‘Wait’ state.
2. Physically connect the CCS cable (simulated by connecting CP/PE pins).
3. Trigger SLAC from the EV side.

**Expected Result:**

- EVSE transitions to ‘SLAC’.
- Matching is successful (GreenPHY association).
- EVSE transitions to ‘Session’ (Protocol Negotiation).

### 5.4.3 TC-03: Authorization

**Objective:** Verify the system correctly processes payment/authorization methods.

**Steps:**

1. Initiate a session with ‘PaymentOption=ExternalPayment’ (EIM).
2. When prompted on the EVSE console, enter "yes" to authorize.

**Expected Result:** EVSE transitions from ‘Authorization’ to ‘Parameter Discovery’.

#### 5.4.4 TC-04: Charging Loop

**Objective:** Verify the main energy transfer loop.

**Steps:**

1. Complete Pre-Charge (Voltage matching).
2. EV requests a current draw (e.g., 10A).

**Expected Result:**

- Contactors close (Simulated).
- EVSE output current ramps to 10A.
- ‘CurrentDemandRes’ messages return ‘DC\_EVSEStatus=Charging’.

#### 5.4.5 TC-05: Session Stop

**Objective:** Verify normal session termination.

**Steps:**

1. Allow SOC to reach target OR manually stop the EV.
2. EV sends ‘SessionStopReq’.

**Expected Result:**

- EVSE initiates Welding Detection.
- Contactors open (Voltage returns to 0V).
- EVSE returns to ‘Wait’ state.

#### 5.4.6 TC-06: Safety/Emergency Stop

**Objective:** Verify immediate shutdown upon fault or unplug.

**Steps:**

1. During ‘Charging’ state, physically disconnect the cable (CP line open).

**Expected Result:**

- EVSE immediately detects ‘CP State A’.
- State transitions to ‘Stop’ or ‘Faulted’.
- Hardware output is disabled (< 20ms).

## Chapter 6

# Reference Documents

- [1] ISO 15118-2:2014. *Road vehicles — Vehicle to grid communication interface — Part 2: Network and application protocol requirements*. International Organization for Standardization.
- [2] ISO 15118-3:2015. *Road vehicles — Vehicle to grid communication interface — Part 3: Physical and data link layer requirements*. International Organization for Standardization.
- [3] IEC 61851-1. *Electric vehicle conductive charging system - Part 1: General requirements*. International Electrotechnical Commission.
- [4] Open Charge Alliance. *Open Charge Point Protocol 2.0.1*.
- [5] Codico. *Whitebeet-EI/PI Hardware Manual*.

# Appendix A

## OCPP 2.0.1 Protocol Details

### A.1 Overview

OCPP 2.0.1 is the latest version of the Open Charge Point Protocol, enabling communication between the EVSE and the Charging Station Management System (CSMS). Unlike previous SOAP-based versions, it relies heavily on JSON over WebSockets for efficient, real-time data exchange. Key improvements include enhanced security, Plug & Charge support, and device management.

### A.2 Core Message Reference

The following table lists the essential messages in the OCPP 2.0.1 protocol used for basic charging sessions and device management.

| Message Name                        | Direction | Description                                                                                      |
|-------------------------------------|-----------|--------------------------------------------------------------------------------------------------|
| <b>Provisioning &amp; Lifecycle</b> |           |                                                                                                  |
| BootNotification                    | CS → CSMS | Sent when the Charging Station boots up to register with the CSMS and get configuration.         |
| Heartbeat                           | CS → CSMS | Periodic signal to indicate the station is still online.                                         |
| Reset                               | CSMS → CS | Commands the station to soft or hard reset.                                                      |
| <b>Authorization</b>                |           |                                                                                                  |
| Authorize                           | CS → CSMS | Verifies a token (RFID, App) to start a transaction.                                             |
| <b>Transactions (Events)</b>        |           |                                                                                                  |
| TransactionEvent (Started)          | CS → CSMS | <b>[NEW]</b> Unified message replacing <i>StartTransaction</i> . Sent when a transaction begins. |
| TransactionEvent (Updated)          | CS → CSMS | <b>[NEW]</b> replaces <i>MeterValues</i> . Periodic updates during charging.                     |
| TransactionEvent (Ended)            | CS → CSMS | <b>[NEW]</b> Unified message replacing <i>StopTransaction</i> . Sent when the transaction stops. |
| <b>Availability &amp; Status</b>    |           |                                                                                                  |
| StatusNotification                  | CS → CSMS | Reports a change in component status (e.g., Connector: Available → Occupied).                    |
| ChangeAvailability                  | CSMS → CS | Requests to change the availability of a connector or station (Operative/Inoperative).           |
| NotifyReport                        | CS → CSMS | <b>[NEW]</b> Reports monitoring data. Part of the new Device Model.                              |
| <b>Remote Control</b>               |           |                                                                                                  |
| RequestStartTransaction             | CSMS → CS | Remotely initiates a transaction (e.g., via mobile app).                                         |
| RequestStopTransaction              | CSMS → CS | Remotely stops an active transaction.                                                            |
| UnlockConnector                     | CSMS → CS | Forces the connector lock to open.                                                               |
| <b>Configuration (Device Model)</b> |           |                                                                                                  |
| GetVariables                        | CSMS → CS | <b>[NEW]</b> Retrieves values of component variables (Replaces <i>GetConfiguration</i> ).        |
| SetVariables                        | CSMS → CS | <b>[NEW]</b> Modifies component variables (Replaces <i>ChangeConfiguration</i> ).                |
| <b>Data &amp; Firmware</b>          |           |                                                                                                  |
| DataTransfer                        | Both      | Exchange of implementation-specific data.                                                        |
| UpdateFirmware                      | CSMS → CS | Instructs the station to download and install new firmware.                                      |
| FirmwareStatusNotification          | CS → CSMS | Reports progress of the firmware update download/installation.                                   |



## Appendix B

# Whitebeet Interface Reference

The `Whitebeet` class encapsulates the low-level communication with the GreenPHY module. Below is a comprehensive list of its public interface methods used for controlling the charging session.

### B.1 System & Configuration

`init(iftyp, iface, mac)` Initializes the interface (Ethernet/SPI) and sets the destination MAC address.

`systemGetVersion()` Retrieves the firmware version of the Whitebeet module (e.g., "1.2.3").

`networkConfigSetPortMirrorState(val)` Configures port mirroring for debugging traffic.

### B.2 Control Pilot (CP)

`controlPilotSetMode(mode)` Sets the operation mode (0: EV, 1: EVSE).

`controlPilotStart()` Starts the CP PWM generation/monitoring.

`controlPilotStop()` Stops the CP service.

`controlPilotSetDutyCycle(cycle)` Sets the PWM duty cycle (e.g., 5.0% for 3A digital communication).

`controlPilotGetState()` Returns the current CP state (A, B, C, D, E, F).

`controlPilotGetVoltage()` Returns the measured voltage on the CP pin.

`controlPilotGetResistorValue()` Returns the measured proximity resistor value.

### B.3 SLAC (GreenPHY)

`slacStart(mode)` Starts the SLAC service (0: EV, 1: EVSE).

`slacStop()` Stops the SLAC service.

`slacStartMatching()` Initiates the SLAC matching process (EVSE side).

`slacSetValidationConfiguration(cfg)` Sets validation parameters for the SLAC association.

## B.4 V2G Session Management (EVSE)

These methods control the high-level ISO 15118 session handling on the EVSE side.

### B.4.1 Configuration

`v2gEvseSetConfiguration(config)` Configures critical EVSE parameters (EVSE-ID, Payment Options, Protocols).

`v2gEvseSetSdpConfig(config)` Configures the SECC Discovery Protocol (ports, security).

`v2gEvseSetSchedules(sched)` Sets the charging schedules (PMax, Sales Tariffs) sent to the EV.

`v2gEvseSetDcChargingParameters(params)` Sets the DC limits (Max Voltage, Max Current, Isolation Level).

`v2gEvseSetAcChargingParameters(params)` Sets the AC limits (Nominal Voltage, Max Current, RCD).

### B.4.2 Session Control

`v2gEvseStartListen()` Starts listening for incoming V2G connection requests (SDP).

`v2gEvseStopListen()` Stops listening for new connections.

`v2gEvseSetAuthorizationStatus(status)` Authorizes the session (e.g., after RFID check).

`v2gEvseSetCableCheckFinished(status)` Signals completion of the Cable Check phase (Isolation test).

`v2gEvseStartCharging()` Signals readiness to start energy transfer (PreCharge/Current Demand).

`v2gEvseStopCharging()` Signals a stop request to the EV.

### B.4.3 Notifications & Parsing

The application must poll for messages using `v2gEvseReceiveRequest()`. Returned payloads are parsed using these helpers:

`v2gEvseParseSessionStarted(data)` Extracts Protocol ID, Session ID, and EVCC-ID.

`v2gEvseParsePaymentSelected(data)` Extracts selected payment method (Contract/PnC).

`v2gEvseParseEnergyTransferModeSelected(data)` Extracts requested energy mode (DC Extended, etc.).

`v2gEvseParseDcChargeParametersChanged(data)` Extracts EV target voltage/current and SOC.

`v2gEvseParseStartChargingRequested(data)` Extracts the charging profile accepted by the EV.

`v2gEvseParseSessionStopped(data)` Extracts the session closure reason.